AI-NATIVE GROCERY MERCHANT

Document 05 — Optimization Decision Specification

Development-only specification • Locked MVP • Basket, quantity, pack, deal, voucher and loyalty optimization

> **Core optimization principle: The AI should not maximize discounts or spend the available budget. It should maximize the user's overall value while satisfying requirements and hard constraints.**

Document status: LOCKED DEVELOPMENT SPECIFICATION

Scope: Requirement fulfillment, product selection, quality-aware quantity/pack optimization, deal optimization, smart incentive decisions, Best Value, Best Quality and recommendation.

1. Purpose

This document defines the deterministic and AI-assisted optimization behavior of the locked grocery merchant MVP. It establishes the inputs, objectives, candidate generation rules, scoring principles, incentive logic, basket construction, trade-off handling and acceptance criteria required to turn a grocery intent into two feasible purchase proposals.

Optimization must remain bounded by the user's mandate and by authoritative backend commerce state. The optimization layer proposes choices; it does not authorize payment.

1.1 Locked optimization outcomes

• A feasible Best Value basket.

• A feasible Best Quality basket, when one exists within the authorized budget.

• A smart incentive decision for applicable voucher/loyalty opportunities.

• A recommendation between the two baskets.

• A structured explanation of material trade-offs.

• A recoverable proposal when a policy check rejects a basket.

2. Optimization Architecture

```
USER GOAL
↓
REQUIREMENTS
↓
CANDIDATE PRODUCTS
↓
QUALITY / EVIDENCE
↓
QUANTITY + PACK CANDIDATES
↓
DEAL OPTIMIZATION
↓
INCENTIVE EVALUATION
↓
BEST VALUE ■■■■■■■■■■
■→ RECOMMENDATION
BEST QUALITY ■■■■■■■■
↓
USER SELECTION
↓
AUTHORITATIVE QUOTE
↓
POLICY ENGINE
```

2.1 Separation of responsibilities

Layer	Responsibility
Agent	Interpret requirements, compare candidates, orchestrate optimization and explain trade-offs.
Catalog	Authoritative product identity, current price, stock and merchant attributes.
Research/Evidence	Approved quality/review evidence used for product evaluation.
Optimizer	Generate and rank feasible quantity/pack/product combinations.
Incentive engine	Deterministically evaluate voucher and loyalty eligibility, savings and constraints.
Cart/quote	Calculate authoritative gross, discount and final payable.
Policy engine	Determine whether the selected proposal is authorized.

3. Optimization Inputs

3.1 Required inputs

Input	Source	Authority
Requirements	Agent extraction	User intent, validated
Budget	Mandate/user authorization	Hard constraint
Allowed category	Mandate	Hard constraint
Max per item	Mandate	Hard constraint
Product facts	Catalog	Authoritative
Stock	Catalog	Authoritative
Quality evidence	Research/evidence store	Evidence authority
Voucher rules	Incentive service	Authoritative
Loyalty rules	Incentive service	Authoritative

3.2 Hard vs soft constraints

Hard constraints cannot be traded away: required category, mandate validity, maximum spend, maximum per-item amount, stock availability, explicit exclusions, and applicable safety/business rules.

Soft objectives can be optimized: price, quality, pack efficiency, savings, convenience and incentive usefulness.

When a hard constraint conflicts with a soft objective, the hard constraint always wins.

4. Requirement Fulfillment

Each basket must satisfy the extracted grocery requirements before it is compared on value or quality.

```
Requirement:
item = eggs
target_quantity = 6
unit = pieces
minimum_quality = acceptable

Basket candidate must satisfy:
covered_quantity >= 6
quality >= minimum_quality
stock = available
category = allowed
item_price <= max_per_item
final_basket_payable <= authorized_budget
```

4.1 Excess quantity

• Prefer the smallest practical excess when exact quantity is unavailable.

• Do not buy extra units solely to reach a voucher threshold.

• Excess may be accepted when it is inherent to a valid pack and materially improves value.

• Any meaningful excess should be visible in the basket explanation.

5. Product Candidate Selection

Candidate generation should first identify products capable of satisfying each requirement, then remove infeasible candidates before optimization.

5.1 Candidate filters

• Category must be compatible with the requirement and mandate.

• Product must be in stock.

• Price must be valid and current.

• Pack quantity must be known.

• Product must not violate explicit user constraints.

• Quality evidence should be available when quality is material.

• Candidates with unsupported or ambiguous critical data should be excluded or marked low confidence.

5.2 Candidate ranking inputs

Ranking should consider effective unit cost, quality evidence, pack efficiency, stock confidence, explicit user preferences and incentive compatibility. Price alone must not dominate when it would materially reduce quality below the acceptable threshold.

6. Quality-Aware Optimization

Quality is part of value, not a decorative explanation added after the cheapest basket is found.

6.1 Quality levels

Level	Meaning
UNACCEPTABLE	Evidence indicates the product fails a required minimum or has a material unresolved issue.
ACCEPTABLE	Evidence supports use for the stated requirement.
GOOD	Evidence indicates reliable quality above the minimum.
PREMIUM	Evidence supports materially higher quality/reliability where relevant.

6.2 Evidence rules

• Do not convert allegations into facts.

• Use evidence references for material quality decisions.

• Conflicting evidence lowers confidence rather than being silently resolved in favor of the cheaper product.

• The system should prefer an evidence-backed alternative when the cheapest option has materially poor evidence.

• Quality claims should be phrased according to evidence strength.

7. Quantity and Pack Optimization

The optimizer must treat packs as combinations rather than assuming a single SKU is always optimal.

7.1 Pack combination objective

For each requirement, generate combinations that cover the target quantity while minimizing unnecessary excess and considering price and quality.

```
For each requirement:
1. Enumerate eligible pack sizes.
2. Generate bounded combinations.
3. Reject combinations with insufficient coverage.
4. Reject unavailable or unauthorized products.
5. Calculate gross cost deterministically.
6. Attach quality evidence.
7. Rank feasible combinations.
8. Pass top candidates to basket optimization.
```

7.2 Egg example

Requirement: 6 eggs. Candidate A is ■36 for six eggs but has materially poor quality evidence. Candidate B is ■42 for six good-quality eggs. Candidate C consists of three two-egg packs at ■6 per egg with good-quality evidence, total ■36. Candidate C should win when all three packs are valid and in stock because it satisfies quantity, preserves quality and matches the lowest feasible cost.

7.3 Optimization constraint

The optimizer must not select a mathematically cheaper combination if it violates the minimum quality requirement. Conversely, it must not select a more expensive product merely because its quality label is higher when the difference is

immaterial to the user's stated need.

8. Deal Optimization

Deal optimization evaluates ordinary merchant pricing and valid bundle/pack benefits before considering vouchers or loyalty rewards.

8.1 Rules

• Use current authoritative catalog prices.

• Compare effective cost per required unit where useful.

• Do not count a discount twice.

• Do not treat a crossed-out/list price as a realized saving unless the backend defines it as such.

• A deal must not cause unnecessary quantity unless the user benefits materially from the additional quantity.

• Final payable must always come from the authoritative quote.

9. Smart Voucher Decision Engine

The incentive engine must answer a value question: should this incentive be consumed now, saved for later, or not used? The largest nominal discount is not automatically the best decision.

9.1 Required evaluation

Factor	Question
Eligibility	Is the user/basket eligible?
Validity	Is the voucher currently valid and usable?
Current saving	How much does the current basket actually save?
Minimum spend	Would reaching the threshold require unnecessary spend?
Expiry	How soon does the benefit expire?
Future utility	Is meaningful future use reasonably expected/known?
Basket dependency	Would using it alter the basket or product choice?
Opportunity cost	Is consuming it now likely to forgo a more valuable use?

9.2 Decision states

State	Rule
USE_NOW	Use when current benefit is meaningful and justified, with no prohibited basket distortion.
SAVE_FOR_LATER	Do not consume when current benefit is small relative to expected future value or the current purchase does not justify i
DO_NOT_USE	Do not use when invalid, ineligible, incompatible, expired, or otherwise harmful.

9.3 Voucher example

A ■100 snack with a 5% voucher saves ■5. If the voucher has meaningful future value, the current ■5 saving is not enough to justify consuming it. The correct decision is SAVE_FOR_LATER. The system should never add unrelated products simply to increase the immediate saving.

9.4 Unknown future value

If future voucher value cannot be reliably established, the engine should use conservative bounded heuristics and expose the uncertainty. It must not invent a future saving.

10. Loyalty Reward Optimization

Loyalty rewards follow the same overall-value principle as vouchers.

• Validate eligibility and redemption rules deterministically.

• Calculate the actual current benefit.

• Do not add unnecessary products to earn points or unlock a tier.

• Prefer preserving rewards when current redemption value is weak and future value is materially better.

• Record the reason for USE_NOW, SAVE_FOR_LATER or DO_NOT_USE.

• Never claim an unavailable reward balance or redemption benefit.

11. Basket Objectives

11.1 Best Value

Best Value minimizes practical total cost subject to requirement satisfaction, acceptable quality and all hard constraints. Practical cost includes valid incentives but must not rely on unnecessary basket expansion.

11.2 Best Quality

Best Quality maximizes practical product quality/reliability subject to requirement satisfaction and the authorized budget. It may cost more than Best Value, but the additional spend must correspond to a meaningful quality improvement.

11.3 Budget interpretation

The authorized budget is a ceiling. Remaining budget is not a reason to add products or upgrade every item. Best Quality may remain significantly below the budget if higher-quality upgrades would not provide sufficient value or no suitable upgrades exist.

12. Basket Scoring Model

The implementation should use a deterministic scoring/ranking layer for repeatability. The exact weights may be tuned during development, but hard constraints must always be evaluated before scoring.

```
Feasible(candidate) =
requirements_met
AND category_allowed
AND stock_available
AND max_per_item_ok
AND quality_minimum_met
AND final_payable <= max_spend

Best Value ranking:
1. feasible
2. lower final payable
3. acceptable/better quality
4. lower excess quantity
5. better incentive usefulness
6. lower uncertainty

Best Quality ranking:
1. feasible
2. higher quality/reliability
3. lower uncertainty
4. lower final payable among materially similar quality
5. lower excess quantity
6. better incentive usefulness
```

The ranking order is more important than an opaque single scalar score. This makes the optimizer explainable and easier to test.

13. Multi-Requirement Basket Optimization

Individual item choices cannot always be optimized independently because vouchers, loyalty rewards and total budget can depend on the complete basket.

13.1 Two-stage approach

• Stage 1: generate bounded high-quality candidates for each requirement.

• Stage 2: combine candidate choices into complete baskets.

• Apply cross-basket constraints and incentive rules.

• Calculate authoritative totals for surviving combinations.

• Select Best Value and Best Quality from the feasible set.

13.2 Search bounds

The MVP should use bounded candidate counts and bounded pack-combination depth. Exhaustive search across a large catalog is out of scope. Candidate generation should reduce the search space before combination.

14. Recommendation Between Baskets

The recommendation layer should identify which basket is the better default for the current user context. It must not erase the alternative.

14.1 Recommendation priority

• Honor explicit user quality preferences.

• Prefer the basket with better requirement satisfaction if one is materially superior.

• Prefer Best Value when quality is acceptable and additional quality cost is not justified.

• Prefer Best Quality when the quality improvement is material and remains comfortably within the mandate.

• Consider incentive preservation when the current saving is weak.

• Expose the trade-off rather than presenting one basket as objectively superior when the difference is preference-dependent.

15. Authoritative Pricing and Arithmetic

The model must never be the source of truth for money arithmetic.

15.1 Calculation ownership

Calculation	Owner
Line-item price	Catalog/backend
Gross subtotal	Cart/commerce service
Voucher discount	Incentive/commerce service
Loyalty benefit	Incentive/commerce service
Final payable	Authoritative cart/quote
Mandate comparison	Policy engine
Razorpay amount	Backend payment integration

The AI may describe amounts returned by these services. It may estimate during reasoning, but any estimate must be discarded before authorization and payment.

16. Re-Optimization After Policy Denial

Optimization must be designed to recover from deterministic rejection without changing the user's authority.

```
Selected basket: ■850
Mandate max: ■800

POLICY → DENY / MAX_SPEND
```

```
Optimizer:
- identify highest-cost removable or replaceable choices
- preserve hard requirements
- preserve minimum quality
- reconsider pack/deal/incentive choices
- generate feasible alternatives
- re-quote
- request policy evaluation again
```

16.1 Recovery constraints

• Do not change max_spend.

• Do not expand allowed categories.

• Do not remove required items silently.

• Do not invent a discount.

• Do not assume stock after a failed stock check.

• Stop after the configured recovery limit and report that no feasible basket was found.

17. Stale Data and Re-Optimization

Catalog, stock and incentive state can change between planning and checkout. Any material state change invalidates the affected optimization result.

Stale condition	Action
Price changed	Recalculate basket and incentives.
Stock lost	Replace affected item or re-optimize.
Voucher expired/invalid	Remove and recalculate.
Loyalty state changed	Re-evaluate reward.
Mandate expired	Stop; do not authorize.
Final quote differs	Use new authoritative quote and re-run policy.

18. Explainability Contract

Every optimized basket must expose concise decision summaries. The system should explain why an item or incentive was chosen without exposing hidden chain-of-thought.

18.1 Required explanation fields

• Requirement satisfied.

• Why the selected product/pack is preferred.

• Quality summary and evidence basis where material.

• Gross amount and final payable.

• Voucher/loyalty decision and actual benefit.

• Key trade-off versus the other basket.

• Any material assumption or uncertainty.

```
{
"decision_summary": {
"reason": "Three 2-egg packs match the required quantity at the same total cost as the 6-pack while using the
better-rated product.",
"evidence_refs": ["evidence-21"],
"uncertainties": []
}
}
```

19. Optimization Data Contract

```
{
"optimization_run_id": "opt-001",
"mandate_id": "mandate-001",
"requirements": [],
"candidate_count": 24,
"best_value": {
"basket_id": "basket-value-001",
"gross_amount": 586,
"discount_amount": 0,
"final_payable": 586
},
"best_quality": {
"basket_id": "basket-quality-001",
"gross_amount": 920,
"discount_amount": 46,
"final_payable": 874
},
"recommendation": {
"basket_id": "basket-value-001",
"reason": "..."
},
"incentive_decisions": [],
"evidence_refs": []
}
```

All monetary fields must use integer minor units internally, such as paise for INR, to avoid floating-point ambiguity.

20. Failure Modes

Failure	Optimization behavior
No feasible candidate	Return unmet requirement; do not fabricate a result.
Only poor-quality candidates	Report quality limitation; do not silently lower the minimum.
Budget too low	Return constrained alternatives or explain infeasibility.
Voucher requires unnecessary spend	SAVE_FOR_LATER / DO_NOT_USE.
Loyalty reward has weak current value	SAVE_FOR_LATER when supported by evidence.
Pack optimization creates excessive quantity	Reject candidate unless excess is justified.
Quality evidence missing	Use only if minimum quality can still be established; otherwise lower confidence or reject.
Policy denial	Re-optimize within unchanged mandate.
Repeated recovery failure	Terminate safely and explain no feasible authorized basket exists.

21. Audit Requirements

Optimization decisions that materially affect the purchase must be traceable.

- Optimization run identifier.
- Input requirement set.
- Candidate/product evidence references.
- Material product substitutions.
- Pack/quantity optimization result.
- Voucher/loyalty decision and rule outcome.
- Best Value and Best Quality basket identifiers.
- Recommendation and reason.
- User-selected basket.
- Policy result.

• Recovery attempts if any.

The audit record should preserve decision facts and references, not hidden model chain-of-thought.

22. Test Scenarios

Scenario	Expected result
Cheap acceptable product vs expensive equivalent	Best Value chooses cheaper; Best Quality may choose higher quality if material.
Cheap poor-quality product vs same-price good pack-quantity option	Poor-quality option rejected; good combination selected.
Voucher saves ■5 with strong future utility	SAVE_FOR_LATER.
Voucher produces meaningful current saving without basket distortion	USE_NOW.
Voucher requires adding unnecessary products	Do not add products; SAVE_FOR_LATER/DO_NOT_USE.
Best Quality would exceed mandate	Must produce a feasible alternative or report infeasibility.
Budget ■1000, basket ■650	Do not add products just to approach ■1000.
Stock disappears after optimization	Refresh and re-optimize.
Policy rejects selected basket at ■850 against ■800 mandate	Re-optimize; mandate remains ■800.
Two baskets have preference-dependent trade-off	Show both and recommend with explicit trade-off; user chooses.

23. Definition of Done

• Requirements are represented as structured optimization inputs.

• Candidate generation respects category, stock and user constraints.

• Quality evidence influences product selection when material.

• Pack and quantity combinations are evaluated rather than assumed.

• Best Value is cost-efficient without violating minimum quality.

• Best Quality maximizes meaningful quality within the authorized budget.

• Budget is treated as a ceiling, never as a spending target.

• Voucher decisions support USE_NOW, SAVE_FOR_LATER and DO_NOT_USE.

• Loyalty decisions follow the same overall-value principle.

• Unnecessary products are never added solely to unlock incentives.

• Authoritative backend services own all money arithmetic.

• Two distinct feasible baskets can be generated for the seeded MVP cases.

• Recommendation is explainable and separate from authorization.

• Policy denial triggers bounded re-optimization without mandate changes.

• Stale price/stock/incentive state causes revalidation.

• Material optimization decisions are auditable.

• All defined test scenarios pass.

• No functionality outside the locked MVP is introduced.

24. Related Development Documents

Document	Relationship
01 — Development Master Spec	Locked product baseline and non-negotiable principles

Document	Relationship
02 — Functional Requirements	Feature behavior and acceptance requirements
03 — System Architecture	Component and trust boundaries
04 — AI Agent Specification	Agent orchestration, tools and reasoning boundaries
06 — Policy & Mandate Specification	Authorization rules used after optimization
07 — Data Model Specification	Persistent optimization, basket and incentive entities
08 — API Contract Specification	Service interfaces for catalog, optimization, incentives and cart
09 — Razorpay Integration Specification	Payment execution boundary
10 — Testing & Acceptance Specification	End-to-end validation